



FABRIC ATLAS

# Fabric Atlas turns Fabric metadata into a shared working catalog.

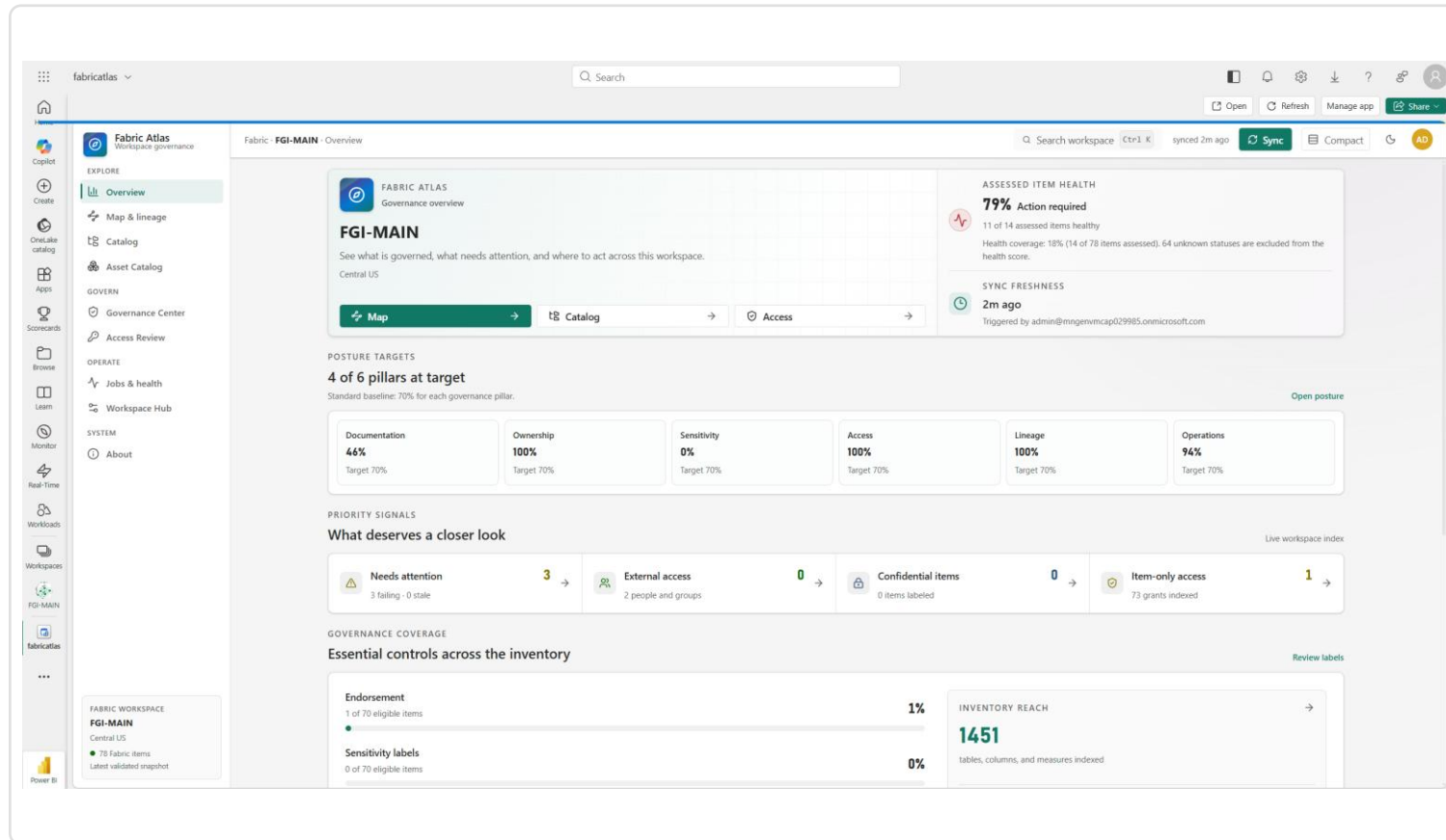
Development, architecture and implementation choices

React + Rayfin · Fabric User Data Functions · Fabric SQL

**Connect → Discover → Map → Persist → Finalize**

12 slides · September 2026 · Fabric Atlas 1.12.4

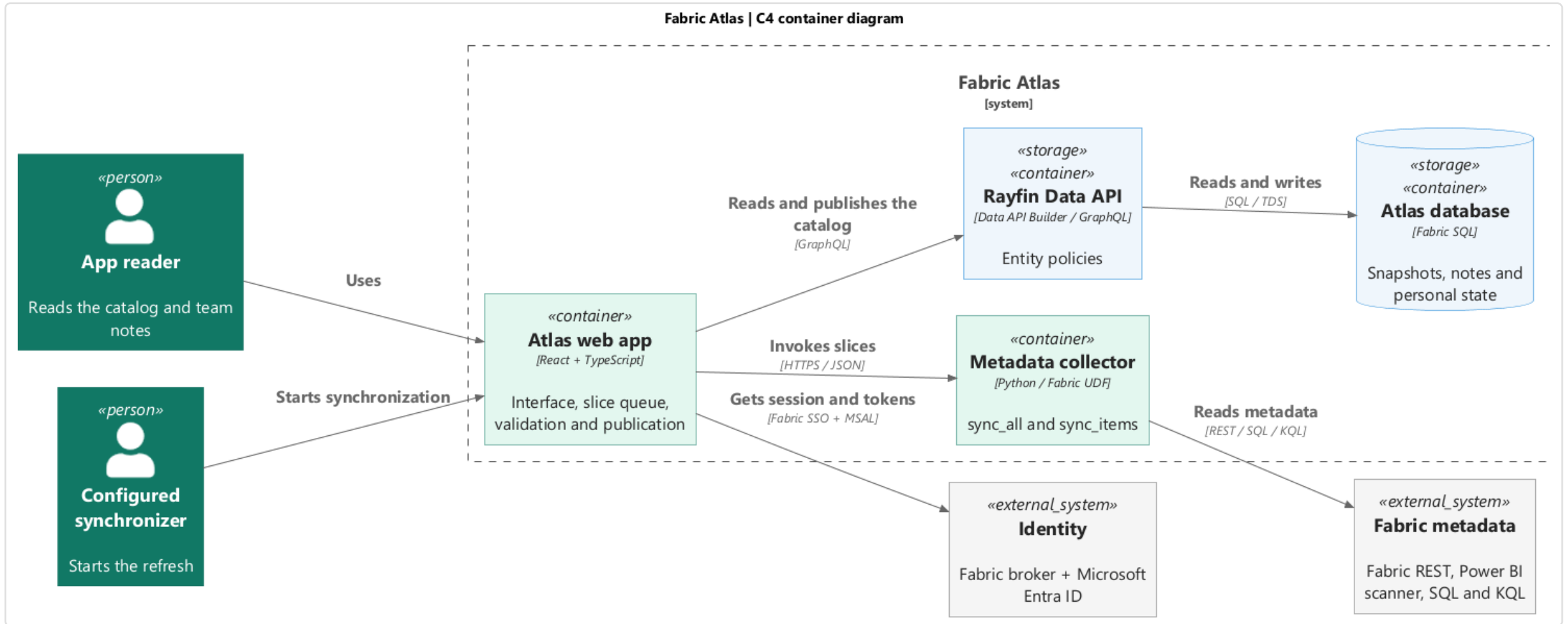
# One validated snapshot powers every view the team needs.



- Catalog of Fabric items and internal objects.
- Lineage, impact, effective access and recent jobs.
- Governance, change history and shared team notes.
- Reads use the Atlas snapshot; pages do not rescan Fabric.

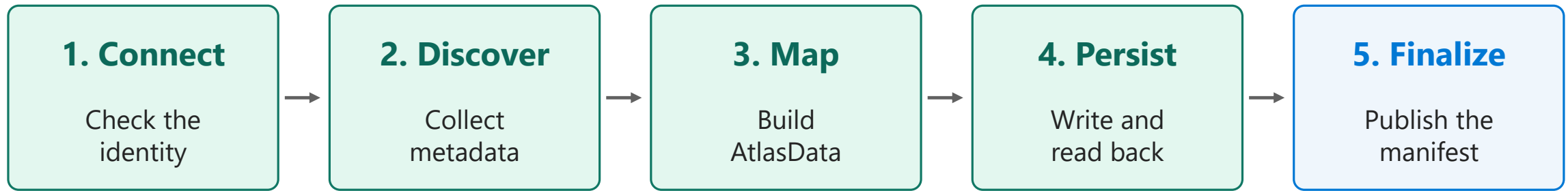
Complete example workspace screenshot; the figures are not product defaults.

# The browser separates Fabric collection from Rayfin publication.



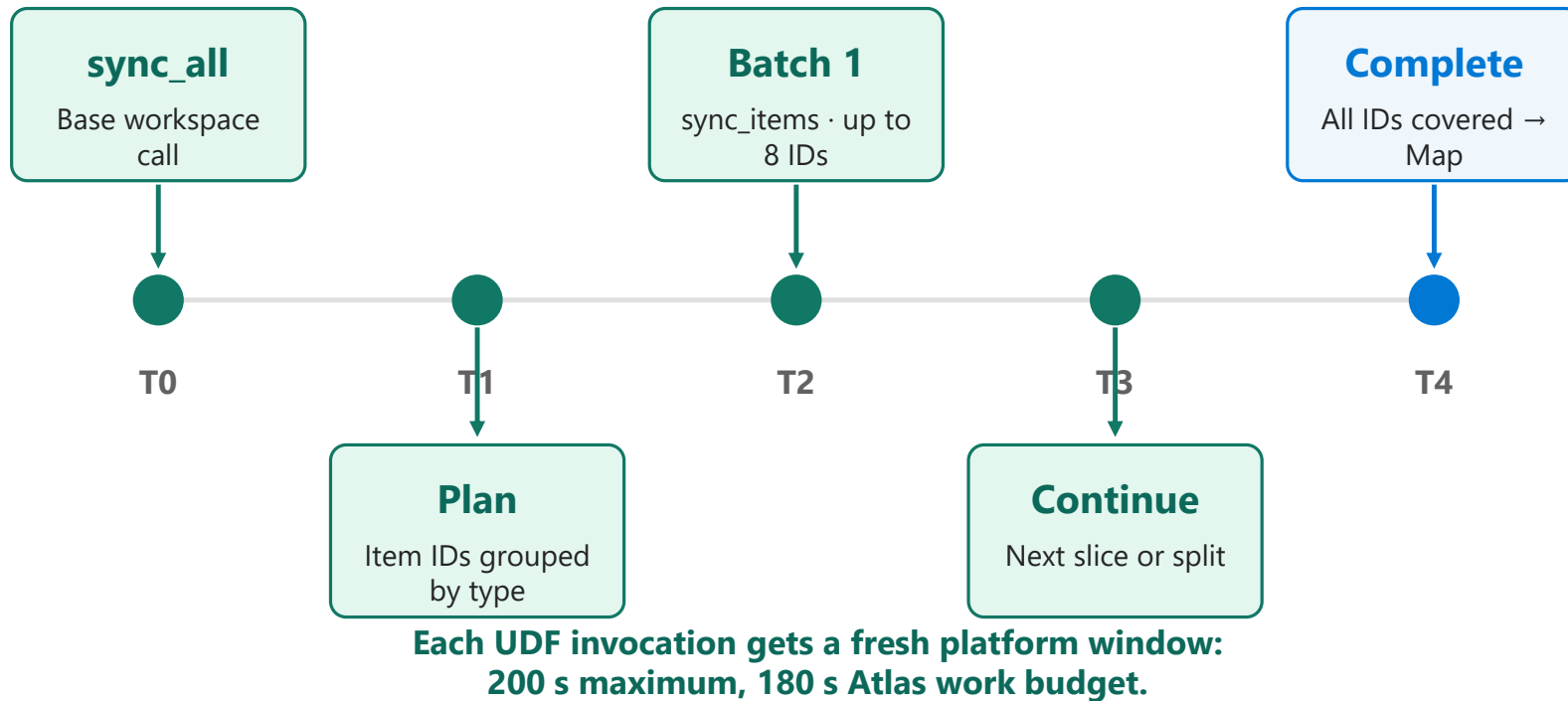
The UDF reads Fabric; the browser validates and publishes through Rayfin.

# Synchronization follows five stages and replaces the catalog only at the end.



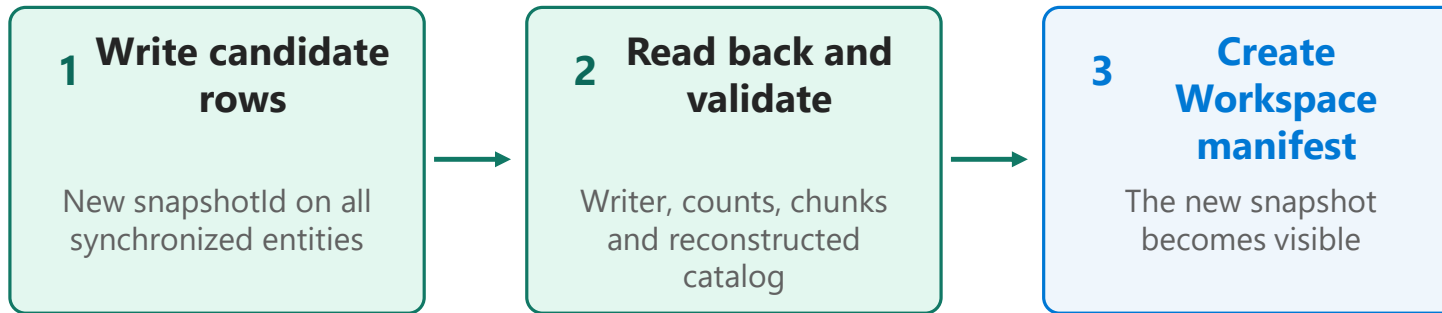
**The previous snapshot stays visible until the Workspace manifest is created.**

# UDF slices process large workspaces without restarting the scan.



- A UDF call has 200 seconds; Atlas uses 180 seconds for work.
- sync\_all returns the base, then sync\_items enriches batches of 8 IDs by type.
- Completed results stay in memory; only remaining IDs return to the queue.

# The Workspace manifest publishes only a complete snapshot that Atlas can read back.



- Child rows use one new snapshotId.
- Read-back uses the same path as application startup.
- Workspace is written last and acts as the visibility switch.

**Failure before the manifest → candidate rows stay invisible and readers keep the previous snapshot.**

# Rayfin receives fifteen entity classes with four different lifetimes.

## Snapshot catalog

Workspace, FabricItem, LineageEdge, Principal, AccessGrant, JobRun and ConfigEntry

## Sync audit

SyncRun records running, completed and failed attempts.

## Shared team state

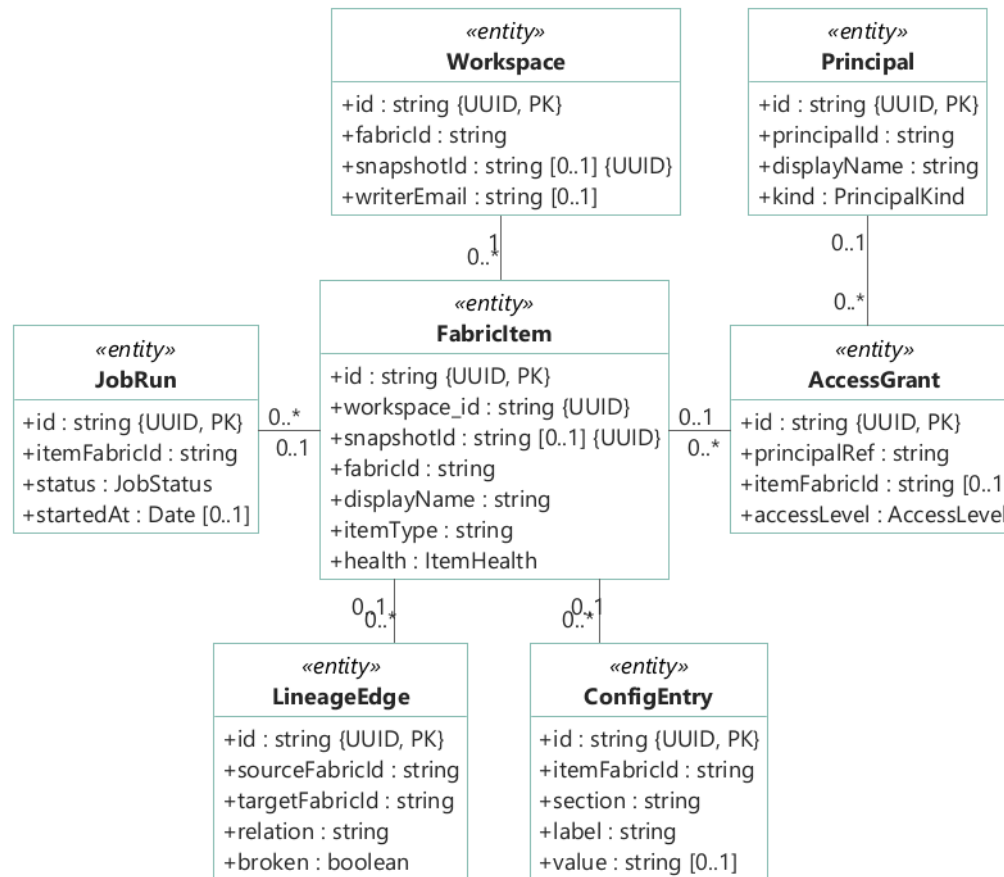
Comment, GovernancePolicy and GovernanceException

## Personal state

SavedView, AccessReview, AccessReviewEvent and FindingAck are filtered by user\_id.

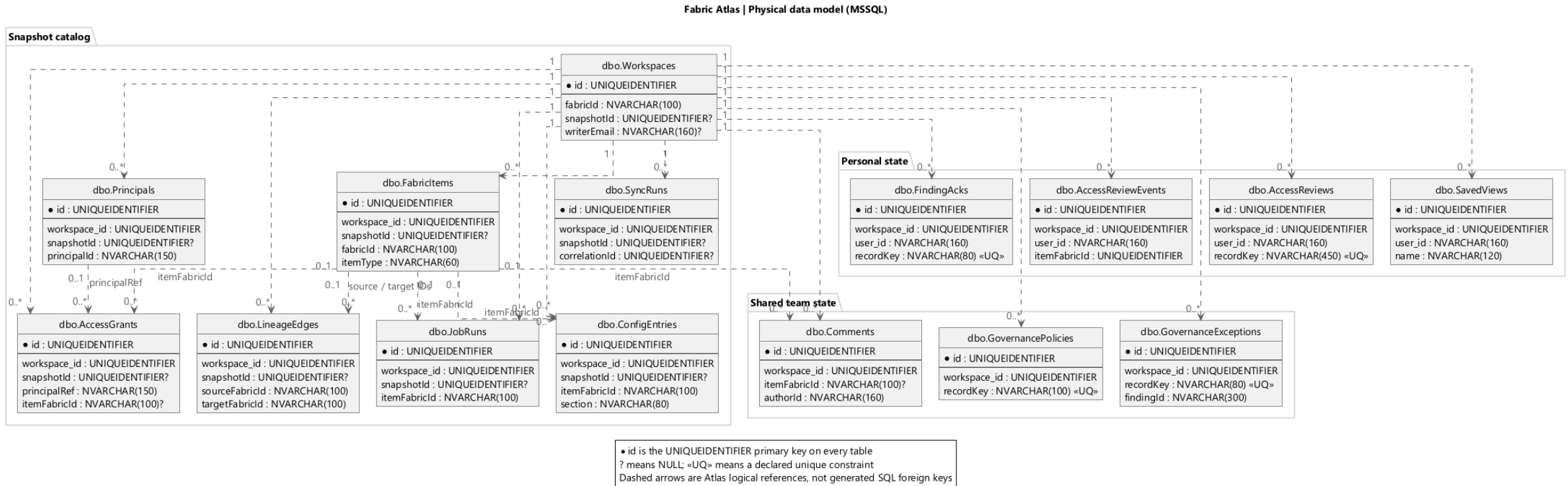
**The entity classes define the database schema and the authorization boundary.**

# The UML model shows entity classes and logical associations.



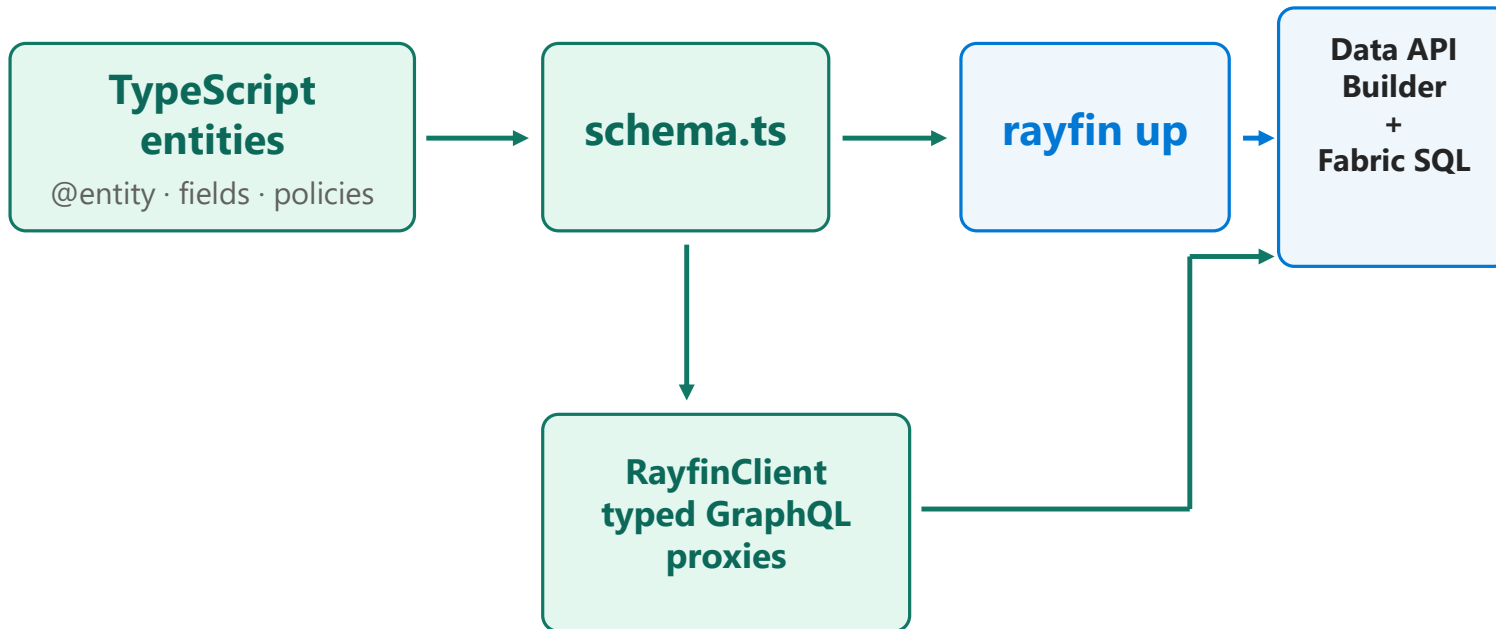
- Selected properties mirror the TypeScript entity classes.
- Workspace publishes a snapshot; FabricItem anchors item-level facts.
- Lines are logical associations, not Rayfin navigation decorators.

# The full MPD shows all 15 MSSQL tables and their logical links.



Rayfin MSSQL mapping 1.33.1 · UNIQUEIDENTIFIER primary keys · NVARCHAR logical references · no generated FK constraints

# Rayfin turns decorated TypeScript classes into the Fabric SQL data model.



- Field decorators define SQL types, limits and optional values.
- Authorization decorators generate read and write policies.
- schema.ts registers the 15 entities used by RayfinClient.

# The flow preserves completed work without publishing partial state.

1. Generic Fabric discovery with no workspace hardcoded.
2. Resumable UDF calls with budgets, continuation and bounded retries.
3. Verified or inferred links are explicit, never invented from names.
4. A snapshot becomes visible only after write, read-back and validation.

Deploy: publish the UDF, then run `npx rayfin up --tenant <tenant-id> --workspace <workspace-name>`



FABRIC ATLAS

# A trusted catalog stays available while the next snapshot is built.

Collect in resumable calls. Model once. Publish only after read-back.

## Questions and feedback

[github.com/fredgis/FabricAtlas](https://github.com/fredgis/FabricAtlas)

C4 · Eraser · Rayfin entities · MCD